Shutong Ji, Swarna Sweth, Aryan Navin, Leyan Huang, Jay Stadelman 9-15-2026
Raz Saremi
Software Engineering I
Fall 2026 CS-GY 6063 I2

Project Ideas

1. Your Nest — an app which displays relevant phone information of your loved ones

TL;DR

There is a lot of information parents and care takers can get from their dependent phones. Location, battery life, recent calls/text, screentime, basically anything we can grab. This information is generally available but it's spread across many apps and subscriptions. Our goal is to integrate as much of it into a single space. To help ensure your children are where they need to be, or that your elderly parents aren't entertaining scam texts and calls.

Project Summary

Project Name	Short Description	Team Enthusiasm Rank (1-10)	Team confidence (1-10)	Product owner
Your Nest	This app integrates all monitoring systems into a single robust dashboard. Monitor battery life, location, wifi connections.	7.8	7	Jay Stadelman

Project Details

Name

Your Nest

Description

This dashboard app will serve as a central hub for monitoring your dependent's phone information. Check in on their location, battery life, wifi connections in real time history. People being monitored may sign up for daily updates on what information was collected from them.

User Personas and Major Features

User Personas:

Guardians: Set up a Nest and select what information is to be collected. Can also update data for usability

Dependants: Allows Your Nest to collect information and receives routine reporting about data

Additional care takers: Can monitor dashboards for safety and updates.

Major Features:

- Dashboard with collected information
- Battery life
- Location
- Wifi connections

MVP (Minimum Viable Product)

- Basic user sign in with different displays depending on authorization level
- Dashboard with relevant collected data
- User ability to turn off monitoring and delete data
- Current and prior location and battery life

MLP (Minimum Loveable Product)

- Alerts for dying phones and unusual locations.
- User to user information sharing via the dashboard/alerts
- Customizable dashboard features – images of users, fun icons for common locations and or high battery

Nice-To-Have Features

- Wifi connections (current and history)
- Step count
- Call/Text logs
- User created tasks – choir check boxes, medicine reminders/status
- Current and future weather
- Calendar integration
- Photo ingestion for user profile pictures and common spaces

CRUD Features by Persona

Guardians

C - Create dependent profiles, invitations, monitoring items, and alert rules
R - Read dependent location, battery, network status, and history
U - Update dependent settings, monitoring preferences, and alert thresholds
D - Remove a dependent, delete stored history, or revoke access

Dependants

C - Create/enroll their device and submit approvals to application
R - Read what information is being collected and which guardians have access
U - Update device permissions, consent settings, and profile information
D - Remove app or revoke app’s access

Care Takers
- C - Create viewer profile to see dashboard, but not be monitored
- R - Read dependent location, battery, network status, and history
- U - Update profile information for dependants where necessary
- D - Remove app

Similar Existing Applications

1. FamilyOrbit – Does everything listed above and more. Tracks location, collects texts, screentime, batterylife and much more. An important limitation appears to be how invasive the app is. To collect ios text and call information users must log into their dependant's iCloud account on a computer and leave it online 24/7 with the FamilyOrbit software running. Also, it doesn't appear to be easy to delete personal information. We will try to distinguish ourselves from this app and others by doing user to user messages and custom built features.

2. Qustodio – similar to FamilyOrbit, tracks location, collects text, screentime, internet activity and much more. Again we will try to distinguish ourselves by making your data easy to delete and allowing users a sandbox of tools to create their own features.

3. Market/Use-Case Gap: The apps on the market I found were geared towards children. They were highly invasive and offered little assurances around data privacy and ownership. We will not only target families with children but also elderly adults who require monitoring and assistance. Our app will aim to be more transparent about what is being collected to the dependants and offer more ownership of the data to the guardian. We will also aim to offer tools for users to create their own features and messages to allow them to tailor their dashboard to fit their needs exactly.

Technical Approach

- Frontend: React, TypeScript, and Capacitor
- Backend: Firebase Cloud Functions and TypeScript or Python
- Database: Firebase's "Firestore" cloud NoSQL databases
- Deployment: Signed APK files (Androids, if allowed only will offer IOS), IOS distribution is either be via App Store (costs money) or direct download via Xcode (free, but limited)
- Authentication: Firebase "Authentication" (Swift, SwiftUI, or Objective-C). Many options for authentication – email+password, social media (IE facebook), phone number

Top risks and Mitigations:

1. **Risk:** Mobile OS restrictions may prevent access to certain device data. iOS and Android may restrict background location, Wi-Fi information, call logs, or text-message metadata.
    - **Mitigation:** Prioritize universally supported features, test permissions early on physical devices, and remove or limit features that cannot be implemented reliably. (Already cut call/text longs to it being locked in IOS 12)
2. **Risk:** Background data collection may be inconsistent. Mobile operating systems can suspend applications or delay scheduled background processes to preserve battery life
    - **Mitigation**: Use background services when available, record the timestamp of every device check-in, and display when data was last updated
3. **Risk:** Sensitive user data creates privacy and security risks.Location could expose users if accounts or databases are compromised.
    - **Mitigation**: encrypt communications and data, use authenticated

guardian-dependent relationships, delete data frequently to limit exposure in case of database compromises.

2. Dialogue scripting tool

TL;DR

A game development tool that lets narrative designers collaborate with gameplay programmers. It allows writers to author branching logic, set conditions, and manage variables interchangeably in plain Markdown or a visual node editor with a built-in playtest tool. It ensures clean Git version control, supports localization via AST line hashing, and exports JSON assets for direct implementation into game engines.

Project Summary

Project Name	Short Description	Team Enthusiasm Rank (1-10)	Team Confidence (1-10)	Product Owner
Dialogue scripting tool	A standalone dialogue node editor and Markdown compiler that exports JSON assets for game engines.	8	8	Shutong Ji

Project Details

Name

Dialogue Scripting Tool

Description

Indie game teams often struggle with friction between narrative designers and programmers. Existing engines have pain points such as a lack of an in-engine playtest tool, using long text files that make branching logic hard to follow, or storing coordinates in dialogue nodes that cause Git merge conflicts.

Dialogue Scripting Tool provides general functions that other narrative tools provide, like variable creation, localization, condition checks, and branching logic. It does so in a standalone environment where narrative designers can write in Markdown or edit visual nodes interchangeably. It features a lightweight way to verify narrative continuity, state tracking, and branching logic. To elevate visualization, it supports hierarchical node grouping and isolates visual layout data in sidecar files for better Git version control. Finally, it will export the data in a JSON file for easy implementation in game engines.

User Personas and Major Features

User Personas:

- Narrative Designer: Writes story branches, organizes visual node graphs, sets conditional dialogue logic, and tests player choice reactions locally.

- Gameplay Programmer: Integrates compiled JSON assets into Unity or Unreal, binding state tracking variables to engine gameplay events.
- Localization & QA Coordinator: Exports, translates, and verifies string catalogs, using automated error checking to catch broken links or missing localization keys.

Major Features:

- Manage complex, non-linear dialogue trees with dynamic player choice reactions and multi-path routing.
- Allow users to create key-value paired data for local and global state and variable tracking across conversations.
- Automatically catches dead-end choices, unassigned variables, broken scene links, and unhandled logic states before runtime.
- Clear visual graph organization with collapsible sub-graph clusters, color-coded node categories, and structured Markdown headers.
- Bi-directional sync between Markdown source files and visual nodes.
- Assigning AST line-hashing string IDs for localization
- Export standardized JSON files for game engine parsers.

MVP (Minimum Viable Product)

- Manually convert edits in the Markdown file to update on nodes graph.
- Support for branching logic, conditional statements, and local/global variable state tracking.
- Sidecar metadata system separating visual nodes data from source text for clean Git version control.
- Compiler producing a JSON AST file.

MLP (Minimum Loveable Product)

- Manually convert edits on the node graph to update the Markdown file.
- Standalone playtest features with live variable state tracking, execution path highlighting, and interactive choice stepping.
- Continuity management tools including checking for missing state triggers and unhandled conditions.
- Deterministic AST line-hashing tool for localization workflows.

Nice-To-Have Features

- Real-time sync between Markdown file and nodes graph.
- Dynamic AST node types for runtime dialogue generation, like unscripted player text inputs and LLM-generated responses.
- Automated generation of placeholder voice-over audio files by using TTS libraries.

CRUD Features by Persona

Narrative Designer

C - Create narrative branches, choice nodes, state variables, and character dialogue.
R - Read story flow diagrams, choice paths, and active state variables.
U - Update narrative text, conditional logic, variable values, and visual canvas layouts.
D - Delete obsolete nodes and unneeded choice branches.

Gameplay Programmer

C - Create custom C#/C++ runtime bindings and engine event mappings.
R - Read compiled JSON AST assets.
U - Update runtime variable bindings and parser configurations.
D - Clear obsolete JSON files.

Localization & QA Coordinator

C - Create JSON translation catalogs based on generated AST string keys.
R - Read untranslated dialogues.
U - Update translated text strings.
D - Delete unused localization string IDs from translation catalogs.

Similar Existing Applications

1. Yarn Spinner - Node-based dialogue tool. It provides text scripting and a graph view. But users need to run playtests inside game engines. Its editor writes node positions into dialogue headers, so it's easy to visualize branching logic but hard to do version control.
2. Ink - Markdown-based dialogue tool. Rich in branching logic, but only displayed in Markdown files; hard to track and visualize branching logic.
3. Twine - Standalone dialogue engine. Can playtest directly in its own engine, but hard to export into other game engines.

Technical Approach

- Frontend: React Flow and TypeScript
- Backend: Python 3.11, FastAPI, and Lark
- Database: File-system based
- Deployment: PyPI packages.

Risks and Mitigations

- **Risk:** Large-scale dialogues can cause severe rendering lag.
    - **Mitigation**: Configure React Flow to only render nodes currently visible inside the user's viewport, and offload auto-layout calculation to the backend.
- **Risk:** Multiple nodes displaying might become a "Spaghetti" graph.
    - **Mitigation**: Allow users to collapse nodes into nested macro-nodes.

3. Sports Hub

TL;DR

A centralized dashboard for your favorite team, region or league. Inside the app users will select who they want to follow and the app will pull together the latest news for the teams and leagues, schedules, and where/how to watch the game. The app will also link out to team or league sites and where to buy tickets. The app itself can be deployed as a Progressive Web App allowing us to bypass app stores and/or Xcode.

Project Summary

Project Name	Short Description	Team Enthusiasm Rank (1-10)	Team confidence (1-10)	Product owner
Sports Hub	This app will serve as a central hub for your favorite sports teams/leagues/regions. It will pull together news, schedule, how/where to watch and local conditions.	8.6	8.2	Jay Stadelman

Project Details

Name

Sports Hub

Description

This PWA will allow users to select favorite teams and leagues to follow. The main landing page will contain scheduling information, current game conditions with live updates, relevant news, and integrate links to relevant external sites (team/league sites, streaming services, and ticket platforms).

User Personas and Major Features

User Personas:

Users: Select teams/leagues to follow, review and interact with dashboard

Sponsors: Create and manage advertisements targeted to relevant sports, leagues, teams, regions, or user interests.

Teams/Leagues: Provide or maintain official information such as schedules, venue details, streaming options, news, announcements, and other team or league updates

Devs: Maintain the app, manage integrations and external data sources, troubleshoot issues, moderate advertisements, and add or update application features

Major Features:

- Dashboard with a collection of information
    - Scheduling
    - Real time game updates
    - Relevant team news, league updates
    - How to watch the game
    - How to purchase tickets

MVP (Minimum Viable Product)

- Basic user sign in and team/league selection
- Dashboard with
    - Team schedules
    - Relevant team news and league updates
- User ability to delete account
- API connections to sports outlets for live game updates, news, and schedule

MLP (Minimum Loveable Product)

- additional dashboard features
    - Link to watch the games
    - Link to purchase tickets
    - Live conditions at the game

Nice-To-Have Features

- Hate watch features (features for teams you really dislike)
    - Updates when they are losing/lose
    - Bad news updates
- Ability to request unsupported teams/leagues and features
- Sort by geographical region to see what sports a region has and to better capture rivalries

CRUD Features by Persona

Users

C - Create an account, follow teams/leagues/regions, save preferences, and create a personalized dashboard.

R - Read schedules, scores, news, weather forecasts, streaming information, advertisements, and other content related to followed teams and leagues

U - Update followed teams, leagues, regions, notification settings, and account preferences

D - Remove followed teams/leagues, clear saved preferences, or delete their account

Sponsors
C - Create targeting criteria and advertisements
R - Read campaign information, and available advertising placements
U - Update advertisement content, targeting criteria, campaign dates, and campaign status
D - Remove advertisements or discontinue campaigns

External data providers (Teams/Leagues)
C - Create schedules, announcements, streaming information, news, and other official updates
R - Read published team or league information and related content
U - Update or modify schedules, venue information, streaming links, and announcements
D - Remove outdated, incorrect, or cancelled information
Admin/Devs
C - Add supported leagues/teams, data integrations, advertisement configurations, and application features
R - Read application data, integration status, error logs, advertisements, and system configuration information
U - Update/maintain external API integrations, fix broken links, modify application settings, update team/league mappings, and maintain application features.
D - Remove obsolete records, broken integrations, and invalid advertisements

Similar Existing Applications

1. theScore: This app offers exactly the same features we are targeting. The biggest downside to this app is the predatory gambling ad and marketing.
2. ESPN: App offers the same feature, but in a more round about method. The dashboard isn't a centralized hub for only your favorites but rather the greater league. Our dashboard will be different by being more tailored to the user's preference instead of a generalized structure.
3. Market/Use-Case Gap: This area seems highly saturated with predatory gambling apps. We will offer a streamlined service which is highly tailored to the user's preferences. There are highly tailored dashboards, but they are surrounded by predatory advertising. There are safe apps without predatory advertising, but they are not highly tailored to the user's preferences.

Technical Approach (PWA)

- Frontend: Next.js with React and TypeScript
- Backend: Node.js and TypeScript. A data pipeline which retrieves data from external sports, news, weather, and streaming APIs.
- Database: Neon PostgreSQL
- Deployment: Vercel will host the Next.js frontend. Neon will host the PostgreSQL database
- Authentication: Neon Auth will manage account registration, login, and user sessions. PostgreSQL Row Level Security will restrict users to their own account information and saved preferences.

Top risks and Mitigations

1. **Risk:** External APIs may become unavailable, expensive, rate-limited, or change their interfaces. Sports Hub depends heavily on third-party sports, news, weather, and streaming data providers
    a. **Mitigation:** limit user request and design the system so alternative APIs can be substituted.
2. **Risk:** Data from multiple sources may be inconsistent or difficult to match. Team names, venue names, league identifiers, schedules, and locations may differ between APIs
    a. **Mitigation:** Create mapping tables that standardize external data before it is displayed to users.
3. **Risk:** Scores, schedules, news, or weather may become outdated. Failed API requests or delayed pipeline jobs could cause the dashboard to display outdated information.
    a. **Mitigation:** Use scheduled data-refresh jobs, track when each record was last updated, removed outdated information when appropriate, retry failed API requests, and display update timestamps when appropriate.